

Settlement Algebra and Verified Constant-Product Invariants for Batch DEX Execution

Dana Edwards

April 19, 2026

Abstract

We present a small, compositional mathematical model for decentralized exchange (DEX) execution and prove key safety invariants for constant-product market makers (CPMMs), including a closed-form expression for the *exact* per-swap increase in the product invariant. The development is mechanized in Lean 4/Mathlib.

The paper has five main layers: (i) an additive *settlement algebra* capturing token flows as a commutative group, with a homomorphism Δ to a scalar “net flow”; (ii) a correctness-by-construction objective for batch auctions maximizing executed volume and surplus in lexicographic order; (iii) a CPMM state-transition model with verified K -monotonicity and an exact “ K -gap” remainder theorem, lifted compositionally to batches and bridged to the settlement/objective layers by a refinement lemma; (iv) a proof-carrying canonical execution bridge that turns runtime winner certificates and generator-coverage evidence into a Lean-checked canonical settlement theorem; and (v) a global settlement mechanism interface theorem that lets future DEX mechanisms inherit canonical settlement by discharging local obligations.

1 Overview and scope

This draft explains the mathematics and formal logic embodied by the Lean files:

- `lean-mathlib/Proofs/SettlementAlgebra.lean`: additive settlement model and the homomorphism Δ .
- `lean-mathlib/Proofs/BatchOptimality.lean`: batch objective (A, B) (volume, surplus) and lexicographic comparison.
- `lean-mathlib/Proofs/CPMMInvariants.lean`: CPMM safety theorems, including the closed-form K -gap.
- `lean-mathlib/Proofs/CPMMSettlement.lean`: bridge from CPMM swaps to settlements.
- `lean-mathlib/Proofs/BatchCPMMUnification.lean`: unification: batch settlement safety, batch optimality additivity, and compositional K theorems for sequential swap batches.
- `lean-mathlib/Proofs/SettlementCanonicalExecution.lean`: proof-carrying canonical execution bridge from runtime list certificates and coverage evidence to canonical settlement execution.
- `lean-mathlib/Proofs/SettlementMechanism.lean`: global settlement mechanism interface theorem.

2 Prior art and contribution

Settlement netting is not new. Payment-system literature has long studied bilateral and multilateral netting, liquidity reduction, final settlement, and the legal and systemic-risk assumptions around netting arrangements [5]. Double-entry bookkeeping and account balances also have known algebraic structure; recent accounting-language work explicitly discusses the algebraic structure of T-accounts and double-entry bookkeeping [3]. Constant-product AMMs are likewise established in DeFi practice, including the Uniswap V2 CPMM model [1]. The phrase “settlement algebra” is also used descriptively in electricity market settlement design documents to refer to settlement formulae and access settlement rules [2]. This paper therefore does not claim first use of the phrase in all domains.

The contribution of this paper is therefore not the claim that netting, double-entry conservation, or constant-product invariants were invented here. The contribution is the *formal DEX synthesis*:

1. encode DEX settlement flows as a small additive algebra with explicit caveats about scalar net-flow versus economic value;
2. connect that algebra to correctness-by-construction batch objectives and exact integer CPMM invariants;
3. mechanize the composition in Lean; and
4. expose a runtime-facing proof-carrying interface: a winner certificate, a generator-coverage proof, and a trace-realization proof together imply canonical settlement execution.
5. package that interface as a mechanism-family theorem: any accepted mechanism family whose members satisfy the obligations inherits the same canonical settlement conclusion.

In short, the paper packages known mathematical ingredients into a verifier-facing settlement calculus for DEX execution. The new part is the proof boundary: runtime search remains untrusted, while small certificates and coverage facts are checked against a Lean-proved theorem.

Important semantic caveat. Throughout, we distinguish:

- *Scalar net-flow* $\Delta(s) = \Delta x + \Delta y$ which adds different token units and is therefore *not* a notion of economic value conservation; and
- the *CPMM invariant* $K = x \cdot y$ which is the standard, economically meaningful conservation law for constant-product pools.

The Lean development makes this distinction explicit and proves K -monotonicity as the primary invariant.

3 Preliminaries

We write $\mathbb{N} = \{0, 1, 2, \dots\}$ and $\mathbb{Z}$ for the integers. For $d \in \mathbb{N}$ with $d > 0$, Euclidean division states that for every $N \in \mathbb{N}$ there exist unique $q, r \in \mathbb{N}$ such that

$$N = dq + r \quad \text{and} \quad 0 \leq r < d, \tag{1}$$

where $q = \lfloor N/d \rfloor$ and $r = N \bmod d$ [4].

4 Settlement algebra

4.1 Definition

Definition 1 (Settlement). *Define the settlement space as*

$$\text{Settlement} := \mathbb{Z} \times \mathbb{Z}.$$

An element $s = (\Delta x, \Delta y)$ represents signed changes to two pool reserves. Positive means inflow to the pool, negative means outflow.

Definition 2 (Composition and identity). *Define composition as componentwise addition:*

$$(\Delta x_1, \Delta y_1) \circ_s (\Delta x_2, \Delta y_2) := (\Delta x_1 + \Delta x_2, \Delta y_1 + \Delta y_2).$$

The identity settlement is $0 := (0, 0)$.

Proposition 1 (Abelian group structure). *$(\text{Settlement}, \circ_s, 0)$ is an abelian group (indeed an `AddCommGroup` in *Mathlib*).*

Proof. Componentwise integer addition is associative and commutative, with identity $(0, 0)$ and inverse $-(\Delta x, \Delta y) = (-\Delta x, -\Delta y)$. $\square$

4.2 Net-flow homomorphism and balance

Definition 3 (Net scalar flow). *Define $\Delta : \text{Settlement} \rightarrow \mathbb{Z}$ by*

$$\Delta(\Delta x, \Delta y) := \Delta x + \Delta y.$$

Proposition 2 (Homomorphism). *Δ is a group homomorphism:*

$$\Delta(s_1 \circ_s s_2) = \Delta(s_1) + \Delta(s_2), \quad \Delta(0) = 0, \quad \Delta(-s) = -\Delta(s).$$

Proof. Immediate from distributivity of addition in $\mathbb{Z}$:

$$\Delta((\Delta x_1 + \Delta x_2, \Delta y_1 + \Delta y_2)) = (\Delta x_1 + \Delta x_2) + (\Delta y_1 + \Delta y_2) = (\Delta x_1 + \Delta y_1) + (\Delta x_2 + \Delta y_2).$$

$\square$

Definition 4 (Balanced settlements). *Call a settlement balanced if $\Delta(s) = 0$. Equivalently,*

$$\text{Balanced} := \ker(\Delta) \subseteq \text{Settlement}.$$

Proposition 3 (Kernel is a subgroup). *Balanced is an additive subgroup of Settlement.*

Proof. Kernels of homomorphisms are subgroups; explicitly: if $\Delta(s_1) = 0$ and $\Delta(s_2) = 0$, then $\Delta(s_1 \circ_s s_2) = 0$, and if $\Delta(s) = 0$ then $\Delta(-s) = 0$. $\square$

Remark 1 (“Safety” as a scalar predicate). The Lean development also defines a *scalar safety* predicate

$$\text{isSafe}(s) \iff \Delta(s) \geq 0.$$

This is closed under composition by the homomorphism property and monotonicity of $+$ on $\mathbb{Z}$, but it should not be conflated with economic safety, since it adds different token units.

4.3 Swap-to-settlement embedding

Definition 5 (Swap settlement). *Given natural amounts $\text{amt}_{\text{in}}, \text{amt}_{\text{out}} \in \mathbb{N}$, define the swap settlement (pool perspective)*

$$\text{swap}(\text{amt}_{\text{in}}, \text{amt}_{\text{out}}) := (\text{amt}_{\text{in}}, -\text{amt}_{\text{out}}) \in \mathbb{Z} \times \mathbb{Z}.$$

Then

$$\Delta(\text{swap}(\text{amt}_{\text{in}}, \text{amt}_{\text{out}})) = \text{amt}_{\text{in}} - \text{amt}_{\text{out}}.$$

In particular, if $\text{amt}_{\text{in}} = \text{amt}_{\text{out}}$ then the settlement is balanced.

5 Batch optimality: maximizing volume and surplus

This section formalizes an objective for batch execution that is *correctness-by-construction*: invalid fills (output below minimum) are unrepresentable.

5.1 Intents and valid outcomes

Definition 6 (Intent). *An intent is a pair $i = (a, m) \in \mathbb{N} \times \mathbb{N}$, where a is the input amount and m is the minimum acceptable output.*

Definition 7 (Valid outcome). *For a fixed intent $i = (a, m)$, a valid outcome is either:*

1. *unfilled, or*
2. *filled(o) for some $o \in \mathbb{N}$ with the proof obligation $o \geq m$.*

This is implemented as a dependent inductive type in Lean (`ValidOutcome i`).

5.2 Objective functions

Definition 8 (Volume and surplus). *For intent $i = (a, m)$ and valid outcome o :*

$$\text{volume}(o) := \begin{cases} 0 & \text{if unfilled} \\ a & \text{if filled} \end{cases} \quad \text{surplus}(o) := \begin{cases} 0 & \text{if unfilled} \\ o - m & \text{if filled with output } o \geq m. \end{cases}$$

Because filled outcomes carry the proof $o \geq m$, the subtraction $o - m$ is well-defined in $\mathbb{N}$ (no underflow).

Definition 9 ((A, B) pair). *Define the objective for a single intent/outcome as*

$$\text{AB}(o) := (\text{volume}(o), \text{surplus}(o)) \in \mathbb{N} \times \mathbb{N}.$$

For a batch (a finite list of intent/outcome pairs), define the batch score as componentwise sum:

$$\text{AB}_{\text{batch}} := \sum_j \text{AB}(o_j).$$

In Lean this is `batchAB`, computed by a left fold with `pairAdd`.

5.3 Lexicographic comparison

Definition 10 (Lexicographic order). For $p = (p_1, p_2), q = (q_1, q_2) \in \mathbb{N} \times \mathbb{N}$, define

$$p \preceq_{\text{lex}} q \iff (p_1 < q_1) \vee (p_1 = q_1 \wedge p_2 \leq q_2),$$

and strict order $p \prec_{\text{lex}} q$ analogously with $p_2 < q_2$.

Proposition 4 (Total preorder). $\preceq_{\text{lex}}$ is reflexive, transitive, and total on $\mathbb{N} \times \mathbb{N}$, and the induced equivalence yields antisymmetry for $\preceq_{\text{lex}}$ on pairs.

Proof. Standard case analysis on the first components; see Lean theorems:

- `lexLe_refl`
- `lexLe_trans`
- `lexLe_total`
- `lexLe_antisymm`

□

6 CPMM model and invariants

We model a constant-product pool with reserves $(x, y) \in \mathbb{N}^2$ as in Uniswap V2 [1].

6.1 Zero-fee swap semantics

Definition 11 (Zero-fee output). Given reserves $(x, y) \in \mathbb{N}^2$ and input $a \in \mathbb{N}$, define the (zero-fee) output as

$$q := \left\lfloor \frac{ya}{x+a} \right\rfloor. \quad (2)$$

The updated reserves are

$$x' := x + a, \quad y' := y - q. \quad (3)$$

Lemma 1 (Output is reserve-bounded). For all $x, y, a \in \mathbb{N}$, the output satisfies $q \leq y$.

Proof. Since $a \leq x + a$, we have $ya \leq y(x + a)$. Dividing by the positive denominator $x + a$ gives $\frac{ya}{x+a} \leq y$, and taking floors yields $q \leq y$. □

6.2 The constant-product invariant and the K -gap

Definition 12 (Constant product). Define $K(x, y) := x \cdot y$.

Theorem 1 (K -monotonicity). Under a zero-fee swap update (2)–(3), the product never decreases:

$$K(x', y') \geq K(x, y).$$

Proof (via the exact K-gap identity). We strengthen the claim by showing the exact difference is a Euclidean remainder. Let $d := x + a$ and $N := ya$. By Euclidean division (1), there exist $q, r \in \mathbb{N}$ with $N = dq + r$ and $0 \leq r < d$. By definition $q = \lfloor N/d \rfloor = \lfloor ya/(x+a) \rfloor$ which matches (2). Then:

$$\begin{aligned} K(x', y') &= (x+a)(y-q) \\ &= dy - dq \\ &= dy - (N - r) \\ &= (x+a)y - (ya) + r \\ &= xy + r. \end{aligned}$$

Thus $K(x', y') - K(x, y) = r \geq 0$. This is mechanized as `k_gap_exact` in `CPMMInvariants.lean`. $\square$

Definition 13 (*K-gap remainder*). Define the per-swap remainder (the K-gap) as

$$r(x, y, a) := (ya) \bmod (x+a). \quad (4)$$

Then the exact K-gap identity is

$$(x+a)(y - \lfloor \frac{ya}{x+a} \rfloor) = xy + r(x, y, a). \quad (5)$$

Corollary 1 (Characterizations). For all $x, y, a \in \mathbb{N}$:

1. $K(x', y') \geq K(x, y)$ (nonnegativity),
2. $K(x', y') < K(x, y) + (x+a)$ (bounded increase, since $r < x+a$),
3. $K(x', y') = K(x, y)$ iff $(x+a) \mid (ya)$ (zero remainder).

Proof. Each follows from (5) and standard properties of mod. See `k_gap_nonneg`, `k_gap_bounded`, `k_gap_zero_iff`. $\square$

6.3 Batches: telescoping the exact K-gap

Definition 14 (Sequential execution). Fix an initial state (x_0, y_0) and a list of inputs $[a_1, \dots, a_n]$. Define the sequence of states by iterating (2)–(3): $(x_i, y_i) \mapsto (x_{i+1}, y_{i+1})$ using input a_{i+1} .

Theorem 2 (Exact telescoping sum). For the sequential execution above,

$$K(x_n, y_n) = K(x_0, y_0) + \sum_{i=0}^{n-1} r(x_i, y_i, a_{i+1}), \quad (6)$$

where r is the per-step remainder (4).

Proof. Induction on n . The induction step applies the single-step identity (5) at state (x_i, y_i) and accumulates remainders. This is mechanized as `executeBatchSwaps_K_gap_sum`. $\square$

Corollary 2 (Batch K-monotonicity). Since each remainder is nonnegative, $K(x_n, y_n) \geq K(x_0, y_0)$ for every batch.

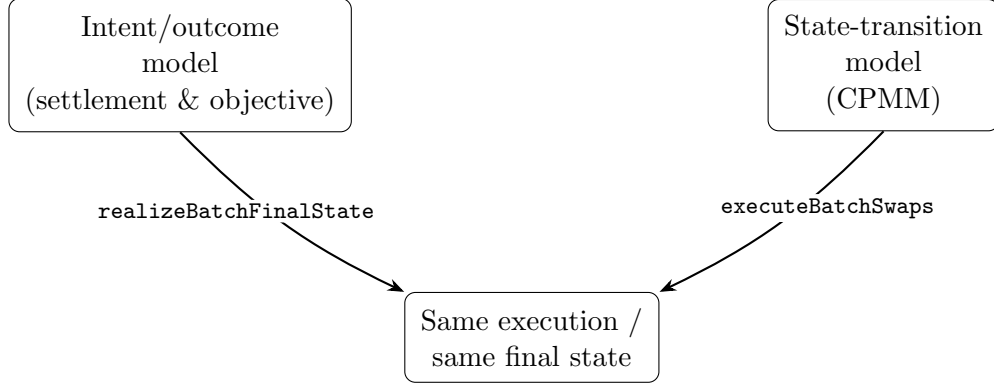


Figure 1: Two views of batch execution that agree by refinement: an intent/outcome view (used for settlement and objective computation) and a reserve state-transition view (used for CPMM invariants).

7 Unification: settlements, objective, and CPMM state

There are two complementary models for batch execution:

1. **Intent/outcome (settlement & objective) model.** A batch is a list of pairs (intent, valid outcome). It induces: (a) an aggregated settlement in `Settlement` (for Δ -analysis) and (b) an aggregated objective score in $\mathbb{N} \times \mathbb{N}$ (for optimization).
2. **State-transition (CPMM) model.** A batch is a list of inputs $[a_1, \dots, a_n]$ executed sequentially on reserves, yielding a final reserve state with verified K properties.

7.1 Aggregating settlements

For each intent/outcome pair, the Lean development defines a per-pair settlement: unfilled contributes 0, and filled contributes $\text{swap}(a, o)$. The batch settlement is the fold (sum) of these settlements. Using the homomorphism property, scalar net-flow composes:

$$\Delta\left(\sum_j s_j\right) = \sum_j \Delta(s_j).$$

Consequently, if each filled execution satisfies $a \geq o$ (a *same-unit* condition), then Δ -safety composes over the batch (`batch_safe_implies_settlement_safe`).

7.2 Refinement bridge and unified safety theorem

The key bridge is that the output-generating function (computing q_i sequentially) reaches the same final state as the pure state-transition fold:

$$\text{realizeBatchFinalState}(s, \vec{a}) = \text{executeBatchSwaps}(s, \vec{a}).$$

This is proved as `realizeBatch_final_state` in `BatchCPMMUnification.lean`.

As a consequence, the CPMM invariants proven for the state-transition model apply to the same execution that feeds settlement/objective computations. The combined statement is captured by `unified_batch_safety`, which simultaneously establishes:

1. $K(\text{final}) \geq K(\text{initial})$,
2. $K(\text{final}) = K(\text{initial}) + \sum \text{remainders as in (6)}$,
3. reserve-in accounting: $x_{\text{final}} = x_{\text{initial}} + \sum_i a_i$.

8 Canonical execution certificates

The newest layer turns the settlement algebra into a runtime-checkable canonical-execution theorem. Let S be a finite set of canonical keys and let w be the selected winner. A finite-set certificate has the form

$$\text{CertificateOK}(S, w) \iff w \in S \wedge \forall x \in S, w \leq x.$$

Standard reading: the winner is in the candidate set and is no worse than every candidate under the explicit total key.

Runtime systems usually emit ordered lists rather than mathematical finite sets, so the Lean development also defines a list-facing certificate:

$$\text{ListCertificateOK}(L, w) \iff w \in L \wedge \forall x \in L, w \leq x.$$

The theorem

`certificateOK_toFinset_of_listCertificate`

proves that this list certificate promotes to the finite-set certificate over $L.\text{toFinset}$. Conversely,

`listCertificateOK_of_certificateOK_toFinset`

lets a verifier check the minimum after deduplicating the emitted list and then recover the list-facing certificate.

The generator-coverage side condition is:

$$\text{CoversKeyList}(L, D, \text{keyOf}) \iff \forall d \in D, \text{keyOf}(d) \in L.$$

Standard reading: every feasible domain object has a key that appears in the runtime-emitted list. The adapter theorem `coversKeyList_of_toFinset_eq_image` handles the common audit shape:

$$L.\text{toFinset} = \text{image}(\text{keyOf}, D) \implies \text{CoversKeyList}(L, D, \text{keyOf}).$$

Putting the pieces together, the main runtime bridge is:

$$\begin{aligned} & \text{ListCertificateOK}(L, \text{candidate.key}) \\ & \wedge \text{CoversKeyList}(L, D, \text{keyOf}) \\ & \wedge \text{candidate.key} \in \text{image}(\text{keyOf}, D) \\ & \wedge \text{Realizes}(\text{trace}, \text{candidate}) \\ & \implies \text{trace executes the canonical settlement over } \text{image}(\text{keyOf}, D). \end{aligned}$$

This is mechanized as:

`realized_runtime_list_certificate_executes_canonical_image_settlement.`

The equality-audit variant:

`realized_runtime_list_certificate_executes_canonical_of_toFinset_eq_image`

removes the abstract coverage premise when the emitted list is proved to deduplicate exactly to the feasible key image.

The practical consequence is important: the runtime generator does not need to be trusted as an optimizer. It must emit a list, a winner certificate, coverage evidence, and a realization proof. Lean then proves that the executed settlement is canonical over the covered feasible domain.

9 Global settlement mechanism theorem

The canonical execution bridge is now packaged as a global theorem schema. A mechanism exposes a finite feasible domain D , a runtime-emitted key list L , a key map $k : D \rightarrow \text{Key}$, a selected candidate c , and an executed trace t . The required obligations are:

$L.\text{toFinset} = k[D]$	exact generator coverage,
$\text{ListCertificateOK}(L, \text{key}(c))$	winner certificate,
$\text{key}(c) \in k[D]$	winner feasibility,
$\text{Realizes}(t, c)$	trace realization.

Standard reading: the runtime generator emitted exactly the feasible keys, the winner is list-minimal under the canonical key, the winner is feasible, and the actual trace is the selected candidate's trace.

The Lean theorem

`global_settlement_algebra`

proves:

$$\begin{aligned} & \text{GeneratorExact}(M) \wedge \text{WinnerCertified}(M) \\ & \wedge \text{WinnerFeasible}(M) \wedge \text{TraceRealizesWinner}(M) \\ & \implies \text{CanonicalSafeSettlement}(M). \end{aligned}$$

Standard reading: any concrete DEX mechanism satisfying those four obligations inherits canonical safe settlement over its feasible key image. This is the global completion point: CPMM, orderbook, routing, perps settlement, and future mechanisms do not reprove canonical settlement; they prove they instantiate the interface.

10 Mechanization notes

The results above are mechanized in Lean 4/Mathlib (no axioms and no `sorry`) [6]. To type-check/build the Lean project:

```
cd lean-mathlib
lake build
```

11 Limitations and future work

This development is deliberately minimal and focuses on arithmetic correctness and composition. Several important directions remain:

- Multi-asset generalization: settlements as $\mathbb{Z}^n$ vectors; richer notions of value conservation.
- Concrete generator completeness: prove that production candidate streams cover the intended feasible domain, or carry a replayable equality/subset audit into `CoversKeyList`.

- Multi-hop routing and cross-pool execution: lifting K -style invariants across routing graphs.
- Overflow-aware models and byte-level constraints matching on-chain execution environments.
- Expanded fee models and protocol-specific mechanics (dynamic fees, rebates, TWAP/oracle guards, etc.).

References

- [1] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap V2. Whitepaper, 2020. Available at <https://uniswap.org/whitepaper.pdf>.
- [2] Australian Energy Market Operator. Optional firm access: Aemo draft report. Technical report, Australian Energy Market Operator, 2014. Uses access settlement algebra terminology in electricity-market settlement design. Available at <https://aemo.com.au/-/media/files/electricity/nem/system-operations/ofa/ofa-draft-report.pdf>.
- [3] Frederico Botafogo. The syntax of the accounting language: A first step, 2019. Available at <https://arxiv.org/abs/1906.10865>.
- [4] Euclid. Elements (Book VII): Euclidean division. *Classical source*, 300. Standard statement: for $N, d \in \mathbb{N}$ with $d > 0$, there exist unique q, r s.t. $N = dq + r$ and $0 \leq r < d$.
- [5] Group of Experts on Payment Systems of the Central Banks of the Group of Ten Countries. Report on netting schemes. Technical report, Bank for International Settlements, February 1989. Available at <https://www.bis.org/cpmi/publ/d02.pdf>.
- [6] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. Lean project, 2023. Reference manual and documentation.